

```

class PuzzleState:
    def __init__(self, board, parent=None, move="", depth=0):
        self.board = board          # Tuple of 9 integers representing the 3x3 grid
        self.parent = parent        # Parent node to reconstruct the path
        self.move = move            # The move made to reach this state
        self.depth = depth          # Current depth in the search tree

    def get_blank_index(self):
        return self.board.index(0)

    def get_neighbors(self):
        neighbors = []
        idx = self.get_blank_index()
        row, col = idx // 3, idx % 3

        # Define valid moves: (row_offset, col_offset, move_name)
        moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

        for dr, dc, move_name in moves:
            new_row, new_col = row + dr, col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_idx = new_row * 3 + new_col
                # Swap the blank space (0) with the adjacent tile
                new_board = list(self.board)
                new_board[idx], new_board[new_idx] = new_board[new_idx], new_board[idx]
                neighbors.append(PuzzleState(tuple(new_board), self, move_name, self.depth + 1))

        return neighbors

def depth_limited_search(state, goal, limit, visited):
    if state.board == goal:
        return state
    if state.depth >= limit:
        return None

    for neighbor in state.get_neighbors():
        if neighbor.board not in visited:
            result = depth_limited_search(neighbor, goal, limit, visited)
            if result is not None:
                return result

    visited.remove(state.board) # Backtrack
    return None

def solve_8_puzzle_dfs(start_board, goal_board):
    """
    Solves the 8-puzzle problem using Iterative Deepening DFS
    to avoid infinite paths inherent to pure DFS.
    """
    start_state = PuzzleState(start_board)
    limit = 0

    while True:
        visited = set()
        solution = depth_limited_search(start_state, goal_board, limit, visited)
        if solution is not None:
            return solution
        limit += 1 # Deepen the search if no solution is found at current depth

def print_solution(solution_state):
    path = []
    current = solution_state
    while current.parent is not None:
        path.append((current.move, current.board))
        current = current.parent
    path.reverse()

    print(f"Initial State:")
    print_grid(current.board)

    for step, (move, board) in enumerate(path, 1):
        print(f"Step {step}: Move Blank '{move}'")
        print_grid(board)
    print(f"Solved in {len(path)} moves!")

def print_grid(board):
    for i in range(0, 9, 3):
        print(f" {board[i]} {board[i+1]} {board[i+2]} ")
    print("-" * 11)

```

```
# --- Example Execution ---
if __name__ == "__main__":
    # 0 represents the empty space
    # Standard easy-to-solve initial state
    initial_configuration = (2,8,3,
                             1,6,4,
                             7, 0,5)

    goal_configuration = (1, 2, 3,
                          8,0,4,
                          7,6,5)

    print("Searching for a solution...")
    solution = solve_8_puzzle_dfs(initial_configuration, goal_configuration)
    print_solution(solution) visited.add(state.board)
```

Searching for a solution...

Initial State:

```
2 8 3
1 6 4
7 0 5
```

Step 1: Move Blank 'Up'

```
2 8 3
1 0 4
7 6 5
```

Step 2: Move Blank 'Up'

```
2 0 3
1 8 4
7 6 5
```

Step 3: Move Blank 'Left'

```
0 2 3
1 8 4
7 6 5
```

Step 4: Move Blank 'Down'

```
1 2 3
0 8 4
7 6 5
```

Step 5: Move Blank 'Right'

```
1 2 3
8 0 4
7 6 5
```

Solved in 5 moves!

Start coding or [generate](#) with AI.